



MLOps Training 2026/2027

TASK #2 FROM TABLES TO NOTEBOOKS

Olist Late Delivery Prediction

MLOps-Oriented Data Preparation, Feature Engineering, Modeling and Evaluation

معالي عبد الرزاق الحاج

Maali Abdul Razaq Elhaj

<https://github.com/mmaali/olist-late-delivery-mlops>

TASK 2 — Olist Late Delivery Prediction

MLOps-Oriented Data Preparation, Feature Engineering, Modeling and Evaluation

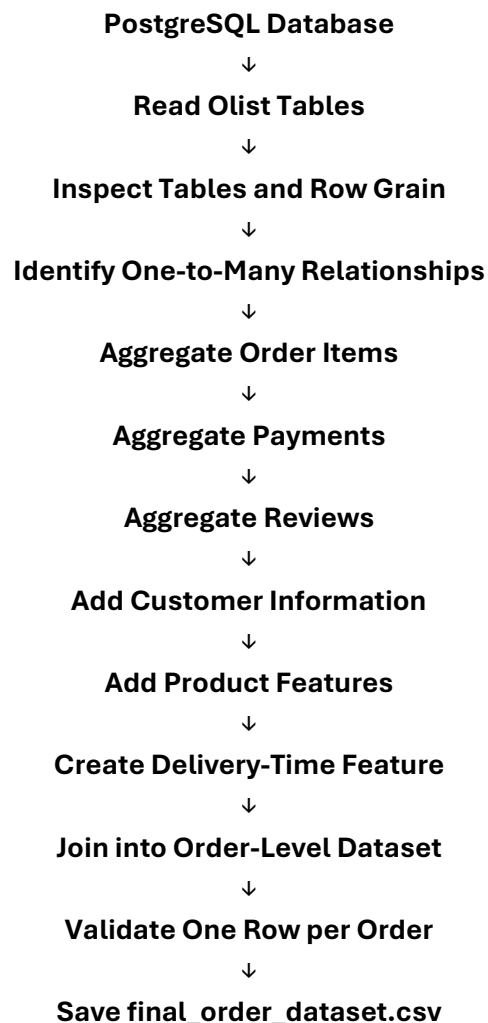
This report documents the complete Task 2 workflow from the order-level Olist dataset through leakage-safe feature engineering, model comparison, threshold optimization, and final evaluation. The work follows the task requirement that each notebook has a clear job, artifacts are passed between notebooks, imbalanced-class metrics are used instead of accuracy alone, and the test set is touched only at the end. `filecite` markers are not included in the report text.

1. Notebook 1 — Read and Join Olist Data

Purpose

The first notebook prepared the Olist data for machine learning by reading the relational tables from the PostgreSQL database running in Docker, inspecting table structure and row grain, aggregating one-to-many tables before joining, and producing one final row per order.

Pipeline



Key Data Preparation

- The orders table was used as the base table because order_id is unique at the order level.
- order_items, order_payments, and order_reviews contain multiple records per order and were therefore aggregated by order_id before joining.
- The resulting dataset included item count, total product price, total freight value, payment aggregates, review aggregates, customer information, product-level aggregates, and delivery-time information.

Final Validation and Artifact

The final order-level dataset contained 99,441 rows and 99,441 unique orders, confirming the required one-row-per-order grain. The prepared dataset was saved as artifacts/final_order_dataset.csv.

2. Notebook 2 — Create the Labels

A delivery label was created by comparing the actual delivery date with the estimated delivery date. Orders delivered after the estimated date were classified as Late, while delivered orders whose actual delivery date was not after the estimate were classified as On Time. A sample of real orders was checked to validate the label logic, and the class distribution was examined to confirm the class-imbalance problem. The labeled dataset was saved as an artifact for the next notebook.

Class Imbalance

Class	Number of Orders
Late	7,826
On Time	88,644
Total	96,470

The strong imbalance between Late and On Time motivated the use of Balanced Accuracy, Late-class Recall, Precision, and F1-score during modeling instead of relying on Accuracy alone.

3. Notebook 3 — Train / Validation / Test Split

The labeled dataset was sorted chronologically using order_purchase_timestamp and divided into 70% Training, 15% Validation, and 15% Test. A time-based split was chosen to better simulate a real deployment setting in which earlier orders are used to develop the model and later orders are used for evaluation. The label distribution was checked across the splits, and train.csv, validation.csv, and test.csv were saved as artifacts.

4. Notebook 4 — Exploratory Data Analysis

EDA was performed using the training split only, keeping validation and test untouched during the exploratory stage. The analysis covered data types, shape, memory usage, missing values, numerical distributions, skewness, outliers, categorical cardinality, group-by comparisons

against the label, date patterns, and geographic variables such as customer state, seller state, and ZIP-code prefix.

Key Findings Relevant to Modeling

- The target is strongly imbalanced toward On Time orders.
- Several numerical variables are right-skewed and contain large ranges, including product weight, freight, price, and payment-related variables.
- Geographical variables showed meaningful structure, motivating the use of customer/seller state and a same_state feature.
- The EDA identified temporal information and geography as promising areas for feature engineering.

5. Notebook 5 — Feature Engineering: Experiment 1

Experiment 1 was designed as the initial leakage-safe modeling pipeline. Features that were available at prediction time were retained, while post-outcome information was removed. Review-related variables and actual delivery duration were excluded because they depend on information that would not be available at prediction time.

Selected Features

- Order/payment information: item_count, total_price, total_freight, payment_count, total_payment, payment_installments.
- Product information: unique_products, average_product_weight, average_product_photos.
- Geography: customer_state, seller_state, customer_zip_code_prefix, seller_zip_code_prefix, and same_state.

Preprocessing

- Numerical variables: median imputation followed by StandardScaler.
- Categorical variables: most-frequent imputation followed by One-Hot Encoding with handle_unknown="ignore".
- The preprocessing transformer was fitted only on the training split and then applied unchanged to validation and test.
- Processed matrices were stored as sparse CSR data to reduce memory usage.

The final Experiment 1 feature representation contained 14 original features and 61 processed features after encoding.

6. Notebook 6 — Modeling: Experiment 1

Evaluation Metrics

Accuracy measures the overall proportion of correctly classified orders. It is reported as a supplementary metric because the dataset is imbalanced.

Late Recall measures the proportion of truly late orders that the model successfully identifies. It is especially important because the system is intended to provide early warning of delayed orders.

Late Precision measures the proportion of orders predicted as Late that are actually Late, indicating the reliability of the model alerts.

Late F1-score balances Late Precision and Late Recall.

Balanced Accuracy gives equal importance to the Late and On Time classes and is therefore appropriate for comparing models under class imbalance.

Models Evaluated

Model	Validation Balanced Accuracy	Late Recall	Late F1
Baseline	0.500	0.000	0.000
Logistic Regression	0.603	0.529	0.146
Random Forest	0.594	0.410	0.150
XGBoost	0.506	0.020	0.030
KNN	0.502	0.010	0.010

Logistic Regression was selected for Experiment 1 because it achieved the highest validation Balanced Accuracy and strong Late Recall among the evaluated configurations. Random Forest was close but lower, while XGBoost and KNN were not competitive for identifying Late orders with the tested settings.

Threshold Optimization

The default Logistic Regression threshold of 0.50 was compared with lower thresholds. Lowering the threshold increased Late Recall but reduced Precision and F1 and, at lower values, reduced Balanced Accuracy substantially. The threshold of 0.50 was therefore retained for the final Experiment 1 evaluation because it provided the best overall balance among the evaluated thresholds.

Experiment 1 Threshold Optimization Results

Threshold	Balanced Accuracy	Late Precision	Late Recall	Late F1
0.50	0.603	0.084	0.529	0.146
0.45	0.602	0.079	0.602	0.139
0.40	0.598	0.072	0.709	0.131
0.35	0.542	0.058	0.928	0.110
0.30	0.519	0.055	0.965	0.105
0.25	0.512	0.055	0.982	0.104

Threshold	Balanced Accuracy	Late Precision	Late Recall	Late F1
0.20	0.504	0.054	0.999	0.102

Interpretation: lowering the threshold substantially increased Late Recall, but reduced Precision and F1-score. The default threshold of 0.50 was retained in Experiment 1 because it provided the highest Balanced Accuracy and the highest Late F1 among the evaluated thresholds.

Experiment 1 Final Test Results

Metric	Experiment 1 Test
Accuracy	65.2%
Balanced Accuracy	46.5%
Late Precision	5.0%
Late Recall	25.0%
Late F1	9.2%

The final test result revealed a clear generalization gap. Although Logistic Regression performed better than the baseline on validation, its Balanced Accuracy fell below 0.50 on the unseen test set. This indicated that the original feature set did not generalize sufficiently well and motivated a focused second experiment.

Experiment 1 Interpretation

- The model could detect some delayed orders but produced a large number of false late-delivery alerts.
- The validation result was not maintained on the unseen test data, indicating that the initial features were not sufficiently informative for robust generalization.
- The result was retained as a legitimate baseline experiment rather than being discarded, because it provides a reproducible reference point for evaluating the value of improved feature engineering.

7. Experiment 2 — Improved Feature Engineering

Experiment 2 was introduced to test whether additional information that is available at prediction time could improve the model without violating the data-leakage rule. The experiment did not rebuild the entire project. It reused the existing train/validation/test artifacts and added a small set of targeted, domain-relevant features.

New Features Added

Feature	Why It Was Added	Leakage-Safe Reason
estimated_delivery_days	Represents the expected delivery window associated with the order.	Uses purchase time and estimated delivery date, not the actual delivery outcome.
approval_delay_hours	Captures the time taken between purchase and approval.	Used under the defined prediction point after order approval and before delivery.
order_day_of_week	Captures weekday-specific operational patterns.	Known from the order purchase timestamp.
order_month	Captures seasonal variation in delivery behavior.	Known from the order purchase timestamp.
is_weekend	Captures potential weekend-related operational effects.	Known at order creation time.
distance_km	Represents the approximate geographic distance between customer and seller.	Calculated from customer and seller ZIP-code geolocation; no delivery outcome is used.

The same_state feature from Experiment 1 was retained. The geographic distance was calculated from the Olist geolocation table by averaging latitude and longitude for each ZIP-code prefix and applying the Haversine formula between the customer and seller coordinates.

Leakage Prevention in Experiment 2

- Actual delivery dates and delivery_days were not used as predictive inputs.
- Review_count and average_review_score were excluded because reviews occur after the order outcome process.
- order_delivered_carrier_date and order_delivered_customer_date were not used as model inputs.
- The transformed pipeline was fitted on training data only and then reused for validation and test.
- The test set was not used to select the Experiment 2 model or threshold; it was evaluated only after the validation-based decision was fixed.

Experiment 2 used 20 original features and produced 84 processed features after preprocessing and one-hot encoding.

8. Experiment 2 — Modeling and Threshold Optimization

Model

Logistic Regression was used as the first and final model for Experiment 2 because it was already the strongest candidate from Experiment 1 and provided a clean way to isolate the effect of improved feature engineering.

Validation Results at the Default Threshold

With the improved feature set and the default threshold of 0.50, Logistic Regression achieved a validation Balanced Accuracy of 0.629, Late Precision of 0.178, Late Recall of 0.351, and Late F1-score of 0.236. Balanced Accuracy, Precision, and F1 improved compared with Experiment 1, while Recall at the default threshold decreased.

Threshold Optimization

Threshold	Balanced Accuracy	Late Precision	Late Recall	Late F1
0.50	0.629	0.178	0.351	0.236
0.45	0.644	0.152	0.419	0.224
0.40	0.659	0.134	0.502	0.211
0.35	0.671	0.116	0.598	0.195
0.30	0.667	0.098	0.690	0.172
0.25	0.656	0.084	0.802	0.153
0.20	0.637	0.075	0.909	0.138

A threshold of 0.35 was selected for Experiment 2 because it produced the highest validation Balanced Accuracy (0.671) while substantially increasing Late Recall to approximately 59%. This was a practical trade-off between capturing more late orders and limiting the loss in overall balanced performance.

9. Experiment 2 — Final Test Evaluation

Metric	Experiment 2 Test
Accuracy	68.7%
Balanced Accuracy	61.2%
Late Precision	11.0%
Late Recall	52.7%
Late F1	18.3%

The final test confusion matrix was $[[503, 454], [4075, 9440]]$. Thus, 503 late orders were correctly identified, while 454 truly late orders were missed. The model also generated 4,075 false late-delivery alerts among orders that were actually On Time.

Most importantly, the improvements observed in validation were also reflected on the unseen test data. This indicates that the additional leakage-safe features improved generalization rather than only fitting the validation split.

10. Experiment 1 vs Experiment 2

Metric	Experiment 1	Experiment 2	Change
Accuracy	65.2%	68.7%	+3.5 pp
Balanced Accuracy	46.5%	61.2%	+14.7 pp
Late Precision	5.0%	11.0%	+6.0 pp
Late Recall	25.0%	52.7%	+27.7 pp
Late F1	9.2%	18.3%	+9.1 pp

Experiment 2 outperformed Experiment 1 across all reported test metrics. The largest improvement was in Late Recall, which increased from 25.0% to 52.7%. Balanced Accuracy also increased substantially from 46.5% to 61.2%, indicating a much more balanced treatment of Late and On Time orders.

11. Feature Interpretation

For the final Logistic Regression model in Experiment 1, the largest absolute coefficients were concentrated in the one-hot encoded `customer_state` and `seller_state` variables. This indicates that geographic context carried substantial predictive signal in the initial feature space. The interpretation does not mean that a specific state causes delay; rather, the encoded state category changes the model score relative to the reference category.

This observation supported the design of Experiment 2, where geographic information was strengthened from simple categorical state labels to a more meaningful quantitative distance feature. The addition of distance therefore followed an interpretable business rationale rather than being a random feature expansion.

12. Limitations and Practical Interpretation

- Late Precision remains low at 11.0% in the final Experiment 2 test result, so many model alerts would still require human or rule-based follow-up.
- The model identifies approximately half of the truly late orders, but it still misses a substantial portion of delays.
- The experiments were deliberately leakage-safe. This limits the model to information that would genuinely be available before the delivery outcome occurs, which can make the task harder than retrospective prediction.
- The final system should therefore be interpreted as an early-warning ranking/classification aid rather than a guarantee of late delivery.

13. Final Conclusion

The Task 2 workflow progressed from relational Olist data preparation to an order-level labeled dataset, chronological train/validation/test splitting, training-only EDA, leakage-safe preprocessing, comparative modeling, threshold optimization, and final evaluation. The first experiment established a reproducible baseline and demonstrated the limitations of the initial feature set. The second experiment introduced targeted temporal and geographic features that were available at prediction time and produced a substantial improvement on the unseen test set.

The preferred final approach is Logistic Regression with the Experiment 2 feature set and a classification threshold of 0.35. Its final test results were 68.7% Accuracy, 61.2% Balanced Accuracy, 52.7% Late Recall, 11.0% Late Precision, and 18.3% Late F1-score. These results represent a clear improvement over Experiment 1, particularly in the ability to identify late orders.

14. Artifacts Produced

- Order-level dataset: artifacts/final_order_dataset.csv
- Labeled and split datasets: artifacts/train.csv, artifacts/validation.csv, artifacts/test.csv
- Experiment 1 preprocessing and feature artifacts: preprocessor.joblib, X_train.npz, X_validation.npz, X_test.npz, y_train.npy, y_validation.npy, y_test.npy, feature_list.txt
- Experiment 2 preprocessing and feature artifacts: preprocessor_experiment2.joblib, X_train_experiment2.npz, X_validation_experiment2.npz, X_test_experiment2.npz, y_train_experiment2.npy, y_validation_experiment2.npy, y_test_experiment2.npy, feature_list_experiment2.txt

- Final Experiment 1 model and results: final_logistic_model.joblib, final_threshold.json, final_results.json
- Final Experiment 2 model and results: final_logistic_model_experiment2.joblib, final_threshold_experiment2.json, final_results_experiment2.json